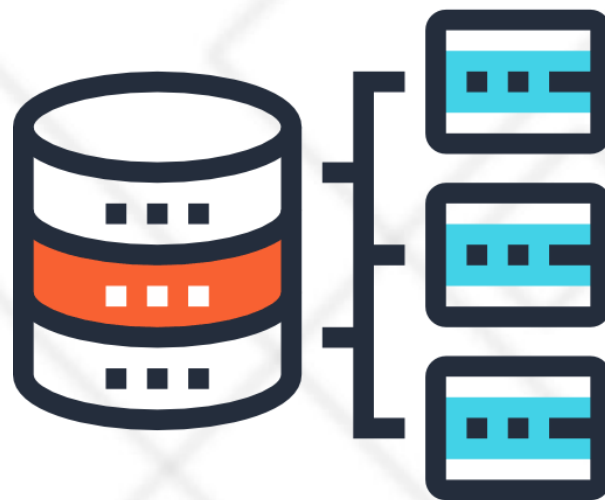




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第12章 数据库编程

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

本章内容



- 12.1 Transact-SQL
- 12.2 游标
- 12.3 存储过程
- 12.4 用户定义函数
- 12.5 触发器

12.1 Transact-SQL

- 数据库的大多数管理操作可以通过标准SQL语言完成
 - 每一种数据库都有各自的特点，必须对标准SQL语言进行扩展才能够操作这些特性
 - 标准SQL语言的非过程性意味着它不能像其它高级语言那样编写可以控制流程（条件判断、循环等）和具有一定结构（函数、子程序等）的程序，数据库软件需要在标准SQL语言的基础上进行扩展
 - ◆ Oracle的PL/SQL
 - ◆ SQL Server的Transact-SQL



12.1 Transact-SQL

- Transact-SQL不仅支持所有的SQL语句，而且还提供了丰富的编程功能，允许使用变量、运算符、函数、流程控制语句等



12.1.1 Transact-SQL元素

□ Transact-SQL元素

- 标识符
- 数据类型
- 常量
- 函数
- 表达式
- 注释
- 保留关键字

12.1.1 Transact-SQL元素

□ 1. 标识符

——符合一定命名规则的标识符

- 第一个字符必须是字母，a ~ z 或 A ~ Z
- 第一个字符后可以是数字、字母或各种符号
- 当标识符的第一个字符是符号时，代表它有特殊用处
 - ◆ 以@开头的标识符代表局部变量或参数
 - ◆ 以@@开头的标识符代表全局变量
 - ◆ 以#开头的标识符代表临时表或存储过程
 - ◆ 以##开头的标识符代表一个全局临时对象

1. 标识符

□ 分隔标识符

- 所有不符合标识符规则的标识符必须进行分隔
 - ◆ 当对象名称包含空格时
 - ◆ 当保留关键字被用作对象名或对象部分的名字时
- 分隔方法
 - ◆ 用中括号 ([]) 进行分隔
 - ◆ 用双引号 (“ ”) 进行分隔

```
SELECT * FROM [My Table]  
WHERE [order]=10;
```

2. 数据类型

- 数据类型是指列、存储过程参数、表达式和局部变量的数据特征，它决定了数据的存储格式，代表了不同的信息类型
- SQL Server提供了各种系统数据类型，还可以自定义数据类型。以下对象可以具有数据类型
 - 表和视图中的列、存储过程中的参数
 - 变量
 - 返回一个或多个特定数据类型数据值的函数
 - 具有一个返回代码的存储过程

系统提供的数据类型



□ 数字数据

- 整型数据：存储整数

bigint	占8个字节，值的范围为 $-2^{63} \sim 2^{63}-1$
int	占4个字节，值的范围为 $-2^{31} \sim 2^{31}-1$
smallint	占2个字节，值的范围为 $-32768 \sim 32767$
tinyint	占1个字节，值的范围为 $0 \sim 255$

- 小数数据：包含存储在最小有效数上的数据

decimal [(p[,s])]	p为精度，最大38； s为小数位数， $0 \leq s \leq p$
numeric [(p[,s])]	在SQL Server中，等价于decimal

系统提供的数据类型

□ 数字数据

- 近似数字数据：表示浮点数据的近似数字

float [(n)]	从 -1.79E+308 到 1.79E+308 之间的浮点数字数据； n 为用于存储科学记数法尾数的位数，同时指示其精度和存储大小， $1 \leq n \leq 53$
real	从 3.40E+38 到 3.40E+38 之间的浮点数字数据，存储大小为4字节； SQL Server 中， real 的同义词为 float(24)

- 货币数据：表示正的或负的货币值，**money**对象的小数点右边最多可以有4个数字

money	占8个字节，值的范围为 -922 337 203 685 477.580 8 ~ +922 337 203 685 477.580 7
smallmoney	占4个字节，值的范围为 -214 748.3648 ~ 214 748.3647

系统提供的数据类型



□ 日期和时间数据

datetime	占8个字节，表示从 1753年1月1日 到 9999年12月31日 的日期
smalldatetime	占4个字节，表示从 1900年1月1日 至 2079年6月6日 的日期

□ 字符数据和Unicode字符数据

char [(n)]	字符个数 0~8 000
varchar [(n)]	字符个数 0~8 000
text	字符个数 0~2GB
nchar [(n)]	字符个数 0~4 000
nvarchar [(n)]	字符个数 0~4 000
ntext	字符个数 0~1GB

系统提供的数据类型



□ 二进制数据

binary [(n)]	存储字节个数 0~8 000
varbinary [(n)]	存储字节个数 0~8 000
image	存储字节个数 0~2G

□ 其他

bit	存储位数据
cursor	存储对游标的引用
rowversion (timestamp)	时间戳
sql_variant	可存储除 text 、 ntext 、 image 、 rowversion 之外的其他类型
table	存储函数返回结果
uniqueidentifier	存储 GUID 以及 UUID

选择数据类型的指导原则

- 若列值的长度相差很大，那么使用变长数据类型
 - 例如某列存储的是人名，地址等
- 谨慎使用tinyint数据类型
 - 虽然节省空间，但扩展性很小
- 对于小数数据来说，一般使用decimal数据类型
 - 可以精确地控制精度
- 如果行的存储量超过8000字节，使用text或image
 - 若不大于8000字节，可使用char、varchar或者binary数据类型
- 对于货币数据，使用money数据类型
- 不要使用类型为float或者real的列作为主键
 - 因为它们不精确，所以不适合用于比较

□ SQL Server提供了用户自定义数据类型的功能

- 用户定义的数据类型与系统数据类型一样，都是用来限制操作者输入数据的种类和长度，它可以用于使用系统数据类型的任何地方
- 如果在model数据库中创建了用户自定义的数据类型，则它将出现在所有以后新建的数据库中。定义在用户数据库中的用户自定义数据类型，则只会出现在定义它的数据库中
- 用户定义的数据类型是与表、视图等并列的数据库对象，可以对它执行创建、修改、删除等操作

用户定义数据类型

□ 用户定义数据类型

- 当多个表的列中要存储同样类型的数据，可使用用户定义数据类型
- 用户定义的数据类型记录在systypes系统表中

□ 创建用户定义的数据类型

- 企业管理器
- 系统存储过程

`sp_addtype {类型名}, [系统数据类型] [, ['NULL'|'NOT NULL']] [, '拥有者']`

□ 删除用户定义的数据类型

`sp_droptype {'类型名'}`

12.1.1 Transact-SQL元素

□运算符

- Transact-SQL提供了如下几种类型的运算符：算术运算符、比较运算符、字符连接运算符和逻辑运算符

- 算术运算符

运算符	含义	运算符	含义
+	加	/	除
-	减	%	取模
*	乘		

- 比较运算符

运算符	含义	运算符	含义
=	等于	!>	不大于
>	大于	!<	不小于
<	小于	<>, !=	不等于
>=	大于或等于	()	控制实行优先级
<=	小于或等于		

12.1.1 Transact-SQL元素

□运算符

- 字符连接运算符

- ◆ 连接运算符只有一种：+，其用于将两个字符数据连接起来

- 逻辑运算符

- ◆ 逻辑运算符一般用于条件判断

□3. 常量

- 常量是表示特定数据值的符号
- 常量的格式取决于它所表示的值的数据类型



3. 常量

常量类型	示例
字符串	'O'Brien' 'Cincinnati'
unicode 字符串	与普通字符串相似，但它前面有一个N标识符： N'Michi'
二进制字符串常量	0x12ef
bit 常量	0 或 1
datetime 常量	'April 15, 1998'、'04/15/98'
integer 常量	1894
decimal 常量	1894.1204、2.0
float 和 real 常量	使用科学记数法表示：0.5E-2
money 常量	以可选小数点和可选货币符号作为前缀的一串数字：\$542023.14
uniqueidentifier 常量	表示全局唯一标识符 (GUID) 值的字符串，可以使用字符或二进制字符串格式： '6F9619FF-8B86-D011-B42D-00C04FC964FF' 0xff19966f868b11d0b42d00c04fc964ff

在Transact-SQL中使用常量



□在Transact-SQL中，可以用多种方式使用常量

□作为算术表达式中的常量值

```
SELECT Price + $.10
```

```
FROM MyTable;
```

□在WHERE子句中，作为与列进行比较的数据值

```
SELECT *
```

```
FROM MyTable
```

```
WHERE LastName = 'O'Brien';
```

□作为将置于变量中的数据值

```
SET @DecimalVar = -1200.02;
```

在Transact-SQL中使用常量



- 设置某列中的数据值

UPDATE MyTable

SET Price = \$99.99

WHERE PartNmbr = 1234;

INSERT INTO MyTable

VALUES (1235, \$88.88);

- 作为指定PRINT或RAISERROR语句发出的消息文本的字符串

PRINT 'This is a message.';

- 作为条件语句（例如：IF语句或CASE函数）中进行测试的值

IF (@@SALESTOTAL > \$100000.00)

EXECUTE Give_Bonus_Procedure;



12.1.1 Transact-SQL元素

□4. 函数：Transact-SQL中提供了大量函数

聚合函数	执行的操作是将多个值合并为一个值。
配置函数	是一种标量函数，可返回有关配置设置的信息。
加密函数	支持加密、解密、数字签名和数字签名验证。
游标函数	返回有关游标状态的信息。
日期和时间函数	可以更改日期和时间的值。
数学函数	执行三角、几何和其他数字运算。
元数据函数	返回数据库和数据库对象的属性信息。
排名函数	是一种非确定性函数，可以返回分区中每一行的排名值。
行集函数	返回可在 Transact-SQL 语句中表引用所在位置使用的行集。
安全函数	返回有关用户和角色的信息。
字符串函数	可更改 char , varchar , nchar , nvarchar , binary 和 varbinary 的值。
系统函数	对系统级的各种选项和对象进行操作或报告。
系统统计函数	返回有关 SQL Server 性能的信息。
文本和图像函数	可更改 text 和 image 的值。

12.1.1 Transact-SQL元素

□5. 表达式

- 表达式是标识符、值和运算符的组合，SQL Server可以对其求值以获取结果

□6. 注释

- 插入到Transact-SQL语句或脚本中、用于解释语句作用的文本段
- SQL Server支持两种类型的注释字符：
 - ◆ 行注释--
 - ◆ 块注释/* ... */

□7. 保留关键字

- 保留下来供SQL Server使用的词

12.1.2 过程的类型

- 可以将Transact-SQL语句以多种方式组合在一起
 - (1) 使用批处理。批处理是作为一个单元从应用程序发送到服务器的一组Transact-SQL语句。每个批处理作为一个可执行单元来执行
 - (2) 使用存储过程。存储过程是在服务器上预定义并预编译的一组Transact-SQL语句。存储过程可以接受参数，并可以将结果集、返回代码和输出参数返回给调用应用程序
 - (3) 使用触发器
 - (4) 使用脚本。脚本是存储在文件中的一系列Transact-SQL语句

1. 批处理

- 一个批处理是由一条或多条T-SQL语句组成的语句集，SQL Server将批中的语句作为一个整体编译为一个执行计划
 - 因为批处理中的语句是一起提交给服务器的，所以可以节省系统开销
 - 在查询分析器中，可以用GO命令标志一个批处理的结束
 - ◆ GO不是一个执行语句，是通知查询分析器有多少语句要包含在当前的批处理中
 - ◆ 查询分析器将两个GO之间的语句组成一个字符串交给服务器去执行

1. 批处理

□批处理有如下限制：

- 一个批处理本身应该是完整的，不能在一个批处理中引用其它批处理定义的变量，也不能将注释从一个批处理开始，在另一个批处理中结束
- 如果批处理中的语句出现编译错误，将不能生成执行计划，批处理中的任何一个语句都不会执行
- CREATE DEFAULT, CREATE PROCEDURE, CREATE RULE, CREATE TRIGGER, CREATE VIEW语句不能与其它语句位于同一个批处理中
- 不能在一个批处理中修改一个表的结构，然后在同一个批处理中引用刚修改的新列

2. 脚本

- 脚本是存储在文件中的一系列Transact-SQL语句
- Transact-SQL脚本包含一个或多个批处理。GO命令表示批处理的结束。如果Transact-SQL脚本中没有GO命令，那么它将被作为单个批处理来执行
- 如果需要经常执行一段Transact-SQL语句或是要进行调试（可能需要反复修改），将执行的T-SQL语句保存到一个脚本文件中可以节省时间

12.1.3 变量和参数

□ Transact-SQL中的变量分为局部变量与全局变量

□ 局部变量

- 局部变量是用户自定义的变量。使用范围是定义它的批、存储过程或触发器。局部变量前面通常加上@标记
- DECLARE定义局部变量，并指明此变量的数据类型
- SET或SELECT命令对其赋值。局部变量的数据类型可以是用户自定义的数据类型，也可以是系统数据类型，但不能将其定义为TEXT或IMAGE数据类型

12.1.3 变量和参数

□ 定义局部变量的语法如下：

**DECLARE @local_variable data_type
[, @local_variable data_type]...;**

- DECLARE命令可以定义多个局部变量，之间用逗号分隔

□ 用SELECT为局部变量赋值的语法如下：

**SELECT @variable_name=expression select statement
[, @variable_name=expression select statement]
[FROM list of tables]
[WHERE expression]
[GROUP BY...] [HAVING...] [ORDER BY];**

- SELECT命令可以将一个表达式的值赋给一个局部变量，也可以将一个SELECT查询的结果赋给一个局部变量

12.1.3 变量和参数

□ 用SET为局部变量赋值

- 用SET为局部变量赋值的常用语法格式为：

SET @local_variable = expression;

□ [例] 使用SET命令赋值的变量

USE MyDb;

GO

DECLARE @no varchar(10);

SET @no='2004060003';

SELECT 编号, 姓名

FROM readers

WHERE 编号=@no;

GO

执行结果为：

编号	姓名
-----	-----
2004060003	李亚茜

12.1.3 变量和参数

- 全局变量是由SQL Server系统提供并赋值的变量
- 在使用全局变量时请注意以下规则：
 - 全局变量是在服务器级定义的，不是由用户例程定义的。用户不能建立全局变量，也不能修改全局变量的值
 - 用户只能使用系统预定义的全局变量
 - 引用全局变量时，前面一定加上@@标记
 - 用户不能定义与系统全局变量同名的局部变量，否则将产生不可预测的结果

12.1.3 变量和参数

□例如， @@ROWCOUNT， 是返回最近一次数据库操作所涉及到的行数

UPDATE Readers SET 已借数量=2;

SELECT @@ROWCOUNT AS 行数;

GO

执行结果为：

行数

5

12.1.3 变量和参数

□ 参数是用于在存储过程和执行该存储过程的批处理或脚本之间传递数据的对象。可以是输入参数，也可以是输出参数

□ [例 12-2] 存储过程的参数

```
USE AdventureWorks;
GO
/* 存储过程ParmSample的参数@EmpIDParm */
CREATE PROCEDURE ParmSample @EmpIDParm int AS
SELECT EmployeeID, Title
    FROM HumanResources.Employee
    WHERE EmployeeID = @EmpIDParm;
GO
/* 批处理中通过参数传递值给存储过程 */
EXEC ParmSample @EmpIDParm = 109;
GO
```


12.1.4 控制流程

□在Transact-SQL程序设计中，流程控制语句用于改变或优化程序的执行顺序，提高执行效率

BEGIN...END	BEGIN 和 END 语句用于将多个 Transact-SQL 语句组合为一个逻辑块。
GOTO	GOTO 语句使 Transact-SQL 批处理的执行跳至标签。
IF...ELSE	IF 语句用于条件的测试。得到的控制流取决于是否指定了可选的 ELSE 语句。
RETURN	RETURN 语句无条件终止查询、存储过程或批处理。
WAITFOR	WAITFOR 语句挂起批处理、存储过程或事务的执行。
WHILE	只要指定的条件为 True 时， WHILE 语句就会重复语句或语句块。
CONTINUE	重新开始 WHILE 循环。
BREAK	终止 WHILE 循环。
CASE	CASE 函数用于计算多个条件并为每个条件返回单个值。

12.1.5 错误处理

- Transact-SQL使用RAISERROR语句生成错误消息并启动会话的错误处理
- sp_addmessage
 - 该存储过程可以用于创建定制的错误消息
 - RAISERROR可以引用sys.messages目录视图中存储的用户定义消息
- @@ERROR
 - 此全局变量返回最近一次系统的错误信息
 - 可以使用局部变量为RAISERROR语句提供消息文本

12.2 Transact-SQL游标

- 应用程序需要一种机制以便每次处理一行或一部分行。游标就是提供这种机制的结果集扩展
- 游标包括
 - 游标结果集：由定义该游标的SELECT语句返回的行的集合
 - 游标位置：指向这个集合中某一行的指针
- Transact-SQL游标主要用在
 - 存储过程、触发器和Transact-SQL脚本中
 - 它们使结果集的内容对其它Transact-SQL语句同样可用

使用游标的典型步骤

- ❑ DECLARE CURSOR 定义 Transact-SQL 服务器游标的特性，例如游标的滚动行为和用于生成游标对其进行操作的结果集的查询
- ❑ OPEN 语句执行 SELECT 语句并填充结果集
- ❑ FETCH 语句提取单个行，并把每列中的数据转移到指定的变量中
- ❑ CLOSE 关闭游标可以释放某些资源，比如游标结果集和对当前行的锁定
- ❑ DEALLOCATE 语句完全释放分配给游标的资源

-- (1) 声明变量以存储由FETCH提取的值

```
DECLARE @LastName varchar(50), @FirstName varchar(50);
```

-- (2) 声明游标contact_cursor

```
DECLARE contact_cursor CURSOR FOR
```

```
    SELECT LastName, FirstName FROM Person.Contact WHERE  
        LastName LIKE 'B%' ORDER BY LastName, FirstName;
```

-- (3) 打开游标

```
OPEN contact_cursor;
```

-- (4) 执行第一次提取并将值存储在变量中

-- 变量的顺序必须与游标SELECT语句中列的个数和顺序相同

```
FETCH NEXT FROM contact_cursor
```

```
    INTO @LastName, @FirstName;
```

-- (5) 检查@@FETCH_STATUS以查看是否有更多的行可以提取

```
WHILE @@FETCH_STATUS = 0  
BEGIN
```

-- (6) 连接并显示变量中当前值

```
PRINT 'Contact Name: ' + @FirstName + ' ' + @LastName;
```

-- (7) 当前一个提取成功时执行下一个提取

```
FETCH NEXT FROM contact_cursor  
INTO @LastName, @FirstName;  
END;
```

-- (8) 关闭并删除游标

```
CLOSE contact_cursor;  
DEALLOCATE contact_cursor;
```



存储过程



- 接受输入参数并以输出参数的格式向调用过程或批处理返回多个值
- 包含执行数据库操作（包括调用其他过程）的编程语句
- 返回状态值，以指明成功或失败（以及失败的原因）
- 可以使用Transact-SQL EXECUTE语句执行存储过程。存储过程与函数不同，因为存储过程不返回取代其名称的值，也不能直接在表达式中使用

存储过程的优点

- 1. 由于在执行前已经存储在数据库中，因此，使用存储过程会比非存储过程获得更好的性能
- 2. 与其他应用程序共享应用逻辑，确保一致的数据访问和修改
 - 存储过程封装了商务逻辑。若规则或策略有变化，则只需要修改服务器上的存储过程，所有的客户端就可以直接使用
- 3. 屏蔽数据库模式的详细资料
 - 用户不需要访问底层的数据库和数据库内的对象

存储过程的优点

□4. 提供了安全性机制

- 用户可以被赋予执行存储过程的权限，而不必在存储过程引用的所有对象上都有权限

□5. 改善性能

- 预编译的Transact-SQL语句，可以根据条件决定执行哪一部分

□6. 减少网络通信量

- 客户端用一条语句调用存储过程，就可以完成可能需要大量语句才能完成的任务，这样减少了客户端和服务端之间的请求/回答包



创建存储过程



- 创建存储过程是用CREATE PROCEDURE语句
 - 可用ALTER PROCEDURE语句进行修改
- 存储过程定义包含两个主要组成部分：
 - 过程名称及其参数的说明
 - 以及过程的主体
- 创建存储过程时，需确定存储过程的组成部分
 - ① 所有的输入参数以及传给调用者的输出参数
 - ② 被执行的针对数据库的操作语句，包括调用其它存储过程的语句
 - ③ 返回给调用者的状态值，以指明调用是成功还是失败



创建存储过程



```
USE Northwind;  
GO  
CREATE PROCEDURE dbo.OverdueOrders  
AS  
    SELECT *  
    FROM dbo.Orders  
    WHERE RequiredDate < GETDATE()  
    AND ShippedDate IS NULL;  
GO
```

- ❑ 只能在当前数据库内创建存储过程，除了临时存储过程。临时存储过程创建在tempdb数据库中
- ❑ 存储过程可以引用表、视图、用户定义函数、其他存储过程以及临时表。若存储过程创建了局部临时表，则当存储过程执行结束后临时表消失
- ❑ 执行CREATE PROCEDURE语句的用户必须是sysadmin、db_owner 或 db_ddladmin 角色的成员，或必须拥有CREATE PROCEDURE权限

执行存储过程



- 使用Transact-SQL EXECUTE语句
- 如果存储过程是批处理中的第一条语句，那么不使用EXECUTE关键字也可以执行存储过程

在存储过程中使用参数

□ 输入参数允许传递信息到存储过程内

- 在CREATE PROCEDURE中指定

@参数名 数据类型 [=默认值]

□ 通过参数名传递值

- 在EXECUTE语句中以“@参数名=值”的格式指定参数称为通过参数名传递
- 可以省略允许空值或具有默认值的参数

□ 输出参数：以OUTPUT关键字指定的变量

- 存储过程通过输出参数向调用它的存储过程或客户端返回信息
- 通过输出参数，存储过程的运行结果可以得到保留，即使存储过程运行结束



12.4 Transact-SQL用户定义函数



- 函数也是一种存储过程，只不过它能返回值，返回值可以是单个标量值或结果集
 - 用户定义函数是系统内置函数的扩展和补充的手段。某些情况下，用户定义函数可以替代视图和存储过程
- SQL Server支持三种用户定义函数：
 - 返回标量值（单值）的函数称为**标量函数**
 - 返回结果集的函数称为**表值函数**
 - ◆ 内嵌 (inline) 表值函数
 - ◆ 多语句表值函数

1. 标量函数

- 标量函数返回一个标量（单值）结果
- 可在与标量函数返回的数据类型相同的值所能使用的任何位置使用该标量函数，包括SELECT语句中列的列表和WHERE子句、表达式、表定义中的约束表达式

标量用户定义函数示例



□ 创建函数

```
USE Northwind;  
CREATE FUNCTION fn_DateFormat  
    (@indate datetime, @separator char(1))  
    RETURNS nchar(20)  
    AS  
    BEGIN  
        RETURN  
        CONVERT(nvarchar(20), datepart(mm, @indate))  
        + @separator  
        + CONVERT(nvarchar(20), datepart(dd, @indate))  
        + @separator  
        + CONVERT(nvarchar(20), datepart(yy, @indate));  
    END;
```

□ 调用

```
SELECT dbo.fn_DateFormat(GETDATE(), ':');
```




表值函数



- 返回table数据类型的用户定义函数功能强大。这些函数称为表值函数
 - 内嵌 (inline) 表值函数
 - ◆ 内嵌表值函数返回一个单条SELECT语句产生的结果的表，使用内嵌表值函数创建参数化视图
 - 多语句表值函数
 - ◆ 多语句表值函数返回一个由一条或多条Transact-SQL语句建立的表

2. 多语句表值函数

- 在Transact-SQL查询中允许使用表或视图表达式
的情况下，可以使用表值用户定义函数
 - 多语句表值函数返回一个由一条或多条Transact-SQL语句建立的表，类似于存储过程
 - 与存储过程不同的是，多语句表值函数可以在SELECT语句的FROM子句中被引用，仿佛视图一样
 - 视图受限于单个SELECT语句，而用户定义函数可包含更多语句，这些语句的逻辑功能可以比视图中的逻辑功能更加强大

多语句表值函数示例



```
USE Northwind;
GO
CREATE FUNCTION fn_Employees (@length nvarchar(9))
RETURNS @fn_Employees TABLE
    (EmployeeID int PRIMARY KEY NOT NULL,
     [Employee Name] nvarchar(61) NOT NULL)
AS
BEGIN
    IF @length = 'ShortName'
        INSERT @fn_Employees SELECT EmployeeID, LastName
        FROM Employees;
    ELSE IF @length = 'LongName'
        INSERT @fn_Employees SELECT EmployeeID,
        (FirstName + ' ' + LastName) FROM Employees;
RETURN;
END;
```

```
SELECT * FROM dbo.fn_Employees('LongName');
```

3. 内嵌表值函数

- 内嵌表值函数返回一个单条SELECT语句产生的结果的表，类似于视图
 - 相对于视图，内嵌表值函数可使用参数，提供了更强的适应性，扩展了索引视图的功能
 - 实际使用内嵌函数时，可以将它看作一个参数化的视图，它一样也可以从中删除和修改数据，最终删除和修改的同样是基表的数据
- 不同于标量函数和多语句表值函数，在内嵌表值函数中只能指定一个RETURNS子句和一个查询，而且不能对返回的表的结构进行定义

内嵌表值函数示例

□ 创建函数

```
USE Northwind;  
GO  
CREATE FUNCTION fn_CustomerNamesInRegion  
  (@RegionParameter nvarchar(30))  
RETURNS TABLE  
AS  
RETURN (  
  SELECT CustomerID, CompanyName  
  FROM Northwind.dbo.Customers  
  WHERE Region = @RegionParameter  
)
```

□ 使用参数调用函数

```
SELECT * FROM fn_CustomerNamesInRegion(N'WA');
```

12.5 Transact-SQL触发器

□创建触发器

- 创建一个触发器时必须指定：
 - ◆ 名称
 - ◆ 在其上定义触发器的表
 - ◆ 触发器将何时激发
 - ◆ 激活触发器的数据修改语句
- 一个表中可有同类型的多个AFTER触发器，前提条件是它们的名称各不相同；每个触发器可以执行多个函数
- 一个表只能有一个给定类型的INSTEAD OF触发器

12.5 Transact-SQL触发器

□和标准SQL的区别

- INSTEAD OF触发器
 - ◆ 先于更新操作执行
 - ◆ 替代原有更新操作的执行
- AFTER触发器
 - ◆ 更新事件发生后执行
 - ◆ 只能定义在表上

触发器示例

□[例] 在books表中建立UPDATE触发器tr1，若更新了books表中的图书编号，则相应更新borrowinf表的图书编号

```
CREATE TRIGGER tr1 ON dbo.books
AFTER UPDATE
AS
    UPDATE borrowinf
    SET 图书编号 = inserted.编号
    WHERE 图书编号 = deleted.编号;
```


触发器示例



□[例] 建立DELETE触发器tr2，当删除readers表中的记录时，若borrowinf表中有相应的借阅记录，则不允许删除该记录

```
CREATE TRIGGER tr2 ON dbo.readers
AFTER DELETE
AS
IF @@ROWCOUNT=0
    RETURN;
IF EXISTS(SELECT * FROM deleted, borrowinf
          WHERE deleted.编号=borrowinf.读者编号)
BEGIN
    PRINT '不允许删除该记录';
    ROLLBACK;
END;
```